

Traivo One - Implementationsförslag för Replit

Dokument: Omfattande gap-analys och implementationsplan

Datum: 2026-06-10

Målgrupp: Replit AI Agent / utvecklare

Källa: `traivo_system_analysis.md` (befintligt GitHub-repo) + `traivo_ui_analysis.md` (önskad funktionalitet)

1. EXECUTIVE SUMMARY

1.1 Nuläge - Traivo One / Traivo Go

Det befintliga systemet i GitHub-repot (`traivo-one`) är en **mobil fältservice-app** (Expo/React Native) med tillhörande Express 5 BFF-server. Appen riktar sig till **förare och tekniker** inom avfallshantering och hanterar:

- Arbetsordrar (visa, starta, slutföra, rapportera avvikelser)
- GPS-spårning och ruttoptimering (Geoapify)
- Inspektionsfoton och digitala signaturer
- Team-vy och brådskande jobb (urgent jobs via Socket.IO)
- AI-assistent (OpenAI) för kontextuell hjälp
- Offline-läge med synkronisering
- Multi-tenant white-labeling

Databasen innehåller **6 tabeller** – alla mobilapp-specifika (push tokens, tidsposter, foton, ruttfeedback, förarpositioner). All domändata (ordrar, kunder, team) ägs av ett bakomliggande Plannix-system och proxas.

1.2 Önskad funktionalitet

Den önskade funktionaliteten (baserad på UI-analys och sessionstranskript) beskriver ett **webbbaserat objekthanteringssystem** med:

- **Hierarkisk objektstruktur** (Koncern → BRF → Fastighet → Rum → Kär!)
- **Dynamiska metadatafält** per objekt
- **Kunddashboard** med KPI:er och processflöde
- **Anpassningsbara listvyer** med valbara kolumner
- **Kalendervy/tidsperspektiv** för uppgifter
- **Kundportal** med rollbaserad filtrering
- **Orderkoncept** (abonnemang, avrop, schema) som ersätter nuvarande ordermodell
- **Felanmälningar som uppgifter** kopplade till objekt

1.3 Identifierade huvudgap

Område	Status	Kommentar
Objekthantering (CRUD + hierarki)	✗ Saknas helt	Ingen objektmodell i DB
Hierarkisk trädvy	✗ Saknas helt	Inget stöd för förälder-barn
Dynamiska metadatafält	✗ Saknas helt	Alla fält är hårdkodade
Kunddashboard + KPI	✗ Saknas helt	Ingen kundöversikt
Kalendervy	✗ Saknas helt	Bara orderlista finns
Kundportal	✗ Saknas helt	Bara fältapp-roller
Orderkoncept (abonnemang/schema)	✗ Saknas helt	Bara enstaka ordrar
Webbgränssnitt (planner/admin)	↻ Rudimentärt	Mockup-sandbox finns men är tom
Multi-tenant datalagring	↻ Delvis	White-label finns men DB saknar tenant-scope
API för objekthantering	✗ Saknas helt	Bara orderrelaterade endpoints

1.4 Rekommenderad prioritering

- Fas 1 (MVP):** Datamodell + Objekthantering + Hierarki + API
- Fas 2 (Förbättring):** Webbgränssnitt + Kunddashboard + Metadata + Listvy
- Fas 3 (Optimering):** Kalendervy + Kundportal + Orderkoncept + Integrationer

2. GAP-ANALYS

2.1 Objekthantering

✓ Vad som redan finns

- Order -modell med ~80 fält (men detta är arbetsordrar, inte "objekt" i den nya definitionen)
- OrderObject -subtyp inom ordrar (innehåller `objectId`, `objectName`, `objectNumber`, `objectType`)
- Mock-data med svenska adresser och realistiska kundnamn
- CRUD-endpoints för ordrar (`GET /my-orders`, `PATCH /orders/:id/status`)

✗ Vad som saknas

- **Objekt som fristående entitet** – idag är "objekt" bara ett fält på en order, inte en egen tabell
- **Objekthierarki** – ingen förälder-barn-relation
- **Objekttyper** (Koncern, BRF, Fastighet, Rum, Kär) – finns inte i schemat

- **Status på objekt** (Aktiv, Arkiverad)
- **Vinjetbild** per objekt
- **CRUD API** för objekt (skapa, läsa, uppdatera, ta bort, flytta)
- **Sökbar trädvy** med aggregerade KPI:er per nod
- **Massåtgärder** (multiselekt i listvy)

Vad som behöver refaktoreras

- `OrderObject` -typen behöver brytas ut till en egen fullständig modell
- `mockData.ts` behöver utökas med objekthierarki-data
- Routing-strukturen behöver nya ruttgrupper för objekthantering

2.2 Hierarki och trädstruktur

✓ Vad som redan finns

- Inget direkt – men `dependencyStatus[]` och `parentWorkOrderId` på ordrar visar att systemet har koncept för relationer
- React Navigation med stack-baserad navigering (kan återanvändas för drill-down)

✗ Vad som saknas

- **Self-referencing foreign key** (`parent_id` → `objects.id`)
- **Rekursiv trädtraversering** (CTE-queries i PostgreSQL)
- **Trädvy-komponent** (kollapsbar/expanderbar med ikoner per nivå)
- **Aggregerade KPI:er** per nod (antal barn, antal kärl, månadsvärde)
- **Drag-and-drop** för att flytta objekt mellan föräldrar
- **Breadcrumb-navigering** i hierarkin

Vad som behöver refaktoreras

- Navigeringsstrukturen behöver stödja deep-linking till specifika hierarkinivåer
- API-arkitekturen behöver stödja rekursiva queries

2.3 Dashboard och KPI:er

✓ Vad som redan finns

- `HomeScreen` med widgets: `NextOrderCard`, `ProgressCard`, `WorkTimeCard`, `WeatherWidget`
- `GET /statistics` – aggregerar tidsposter per dag/vecka/månad
- `GET /summary` – daglig sammanfattning
- `DayReportScreen` – dagrapporter

✗ Vad som saknas

- **Kunddashboard** – översiktssida per kund med KPI-kort
- **KPI-beräkningar**: Antal objekt, aktiva ordrar, månadsvärde, snitt ställtid
- **Processflödesvisualisering** (Objekt → Abonnemang → Ordor → Utförda → Fakturerade)
- **“Räkna om arv”**-funktion (kaskadberäkning genom hierarki)
- **Ekonomiska KPI:er** (månadsvärde, årsdiagram per månad)
- **Planner-dashboard** – webbaserad översikt

Vad som behöver refaktoreras

- `StatisticsScreen` bör utökas med objektbaserade KPI:er
 - `mockup-sandbox` (Vite + Radix UI) kan bli grunden för webbdashboard
-

2.4 Metadata och dynamiska fält

Vad som redan finns

- Inget direkt stöd för dynamiska metadata
- `executionCodes[]` på ordrar visar viss flexibilitet
- `orderNotes[]` för fritext-anteckningar

Vad som saknas

- **Metadatafält-modell** (typ: alfanumerisk, numerisk, bild, fil)
- **Metadataschema-definition** per objekttyp/tenant
- **Dynamiskt formulär** som renderar metadatafält baserat på schema
- **Metadata i listvyer** som valbara kolumner
- **Sökning och filtrering** på metadatavärden
- **Metadata-arv** uppåt/nedåt i hierarkin

Vad som behöver refaktoreras

- Listyns kolumndefinitioner behöver bli dynamiska istället för hårdkodade
-

2.5 Kalender och tidsperspektiv

Vad som redan finns

- `scheduledDate`, `scheduledStart/End`, `scheduledTimeStart/End` på ordrar
- `TodoScreen` - uppgiftslista
- `estimatedDuration` per order

Vad som saknas

- **Kalendervy-komponent** (År → Kvartal → Månad → Vecka → Dag)
- **Expanderbar tidslinje** med zoomning
- **Koppla uppgifter till objekt i kalendervy**
- **SlotPreference** per objekt (tidsfönsterpreferenser)
- **Leveranstid start/slut** som visuell spann i kalender

Vad som behöver refaktoreras

- `TodoScreen` kan utökas med kalenderläge
 - Befintlig schemaläggningsdata kan användas som grund
-

2.6 Orderkoncept och uppgifter

Vad som redan finns

- Fullständig `Order`-modell med livscykelhantering
- Statusövergångar (17 statusar)

- Tidsspårning (`time_entries`)
- Avvikelseberapportering (`deviations`)
- Signatur och kundkvittering

✗ Vad som saknas

- **Orderkoncept** som separat entitet (abonnemang, avrop, schema)
- **Beställningsreferens** kopplad till orderkoncept
- **Uppgift** som fristående entitet (skiljs från "order")
- **Many-to-many** relation orderkoncept ↔ objekt
- **Felanmälan som uppgiftstyp**
- **Dubbelriktad navigering**: Objekt ↔ Uppgifter ↔ Orderkoncept

🔄 Vad som behöver refaktoreras

- `Order` -modellen behöver utökas med orderkoncept-referens
- Statusmodellen behöver anpassas (Planerad → Aktiv → Utförd → Fakturerad)
- Mock-data behöver orderkoncept-exempel

2.7 Kundportal och roller

✓ Vad som redan finns

- Rollsystem: `technician`, `planner`, `admin`, `owner`, `user`
- Blockerade roller: `customer`, `reporter`, `viewer`
- `UnauthorizedScreen` för obehöriga roller
- Multi-tenant via `tenantId`

✗ Vad som saknas

- **Kundportal** – filtrerad vy per kundanvändare
- **Rolltyp** `customer` som tillåts se begränsad data
- **Koppling användare ↔ objekt** (vilka objekt ser en kund)
- **Push-notiser till kunder** för tilldelade uppgifter
- **Kundspecifika kontaktpersoner** per objekt

🔄 Vad som behöver refaktoreras

- `FIELD_APP_BLOCKED_ROLES` – `customer` bör tillåtas med begränsad åtkomst
- `AuthContext` behöver stödja kundportal-läge
- Navigeringsträdet behöver branching baserat på roll

2.8 Webbgränssnitt

✓ Vad som redan finns

- `mockup-sandbox` – Vite + React + Radix UI + Tailwind + Recharts (tomt skal)
- Planner-endpoints (`GET /v1/planner/drivers/locations`, `GET /v1/planner/orders`)
- Planner-autentisering (HMAC-baserad)

✗ Vad som saknas

- **Fullständigt webbgränssnitt** för administration

- **Objekthanteringsvyer** (hierarki, lista, karta, detalj)
- **Kunddashboard-sidor**
- **Kalenderkomponenter**
- **Admin-panel** för metadata-schema-hantering



Vad som behöver refaktoreras

- `mockup-sandbox` kan vidareutvecklas eller ersättas med ny webb-artefakt
- Planner-rutter behöver utökas massivt

3. DETALJERADE IMPLEMENTATIONSFÖRSLAG

3.1 Objekthantering (CRUD)

Prioritet: ● **Hög (Fas 1)**

Affärskrav

- Skapa, läsa, uppdatera och ta bort objekt
- Varje objekt har namn, objektnummer, objekttyp, status, adress, vinjetbild
- Objekt kan ha en förälder (self-referencing)
- Objekt tillhör en tenant

Teknisk lösning - Backend

Drizzle-schema (`lib/db/src/schema/objects.ts`):

```
import { pgTable, uuid, text, varchar, timestamp, pgEnum, index, jsonb } from
'drizzle-orm/pg-core';

export const objectStatusEnum = pgEnum('object_status', ['active', 'archived',
'draft']);
export const objectTypeEnum = pgEnum('object_type', [
'koncern', 'brf', 'fastighet', 'rum', 'karl', 'custom'
]);

export const objects = pgTable('objects', {
  id: uuid('id').defaultRandom().primaryKey(),
  tenantId: varchar('tenant_id', { length: 100 }).notNull(),
  name: varchar('name', { length: 500 }).notNull(),
  objectNumber: varchar('object_number', { length: 100 }).notNull(),
  objectType: objectTypeEnum('object_type').notNull().default('custom'),
  status: objectStatusEnum('status').notNull().default('active'),
  parentId: uuid('parent_id').references(() => objects.id, { onDelete: 'set null' }),

  // Adress
  streetAddress: varchar('street_address', { length: 500 }),
  postalCode: varchar('postal_code', { length: 20 }),
  city: varchar('city', { length: 200 }),
  latitude: text('latitude'),
  longitude: text('longitude'),

  // Bild
  vignetteImageUrl: text('vignette_image_url'),

  // Metadata
  createdAt: timestamp('created_at').defaultNow().notNull(),
  updatedAt: timestamp('updated_at').defaultNow().notNull(),
  createdBy: varchar('created_by', { length: 100 }),
  updatedBy: varchar('updated_by', { length: 100 }),
}, (table) => [
  index('idx_objects_tenant').on(table.tenantId),
  index('idx_objects_parent').on(table.parentId),
  index('idx_objects_type').on(table.objectType),
  index('idx_objects_status').on(table.status),
  index('idx_objects_number').on(table.objectNumber),
  index('idx_objects_tenant_parent').on(table.tenantId, table.parentId),
]);
```

API-endpoints (artifacts/api-server/src/routes/objects.ts):

// GET /api/v1/objects	– Lista objekt (med filtrering, paginering, sökning)
// GET /api/v1/objects/:id	– Hämta enskilt objekt med relationer
// POST /api/v1/objects	– Skapa nytt objekt
// PUT /api/v1/objects/:id	– Uppdatera objekt
// DELETE /api/v1/objects/:id	– Ta bort objekt (soft delete → archived)
// GET /api/v1/objects/:id/children	– Hämta barnobjekt
// GET /api/v1/objects/:id/ancestors	– Hämta hela föräldrakedjan (breadcrumb)
// PATCH /api/v1/objects/:id/move	– Flytta objekt till ny förälder
// GET /api/v1/objects/tree	– Hämta hela trädet (rekursivt, med aggregering)
// GET /api/v1/objects/search	– Fritextsökning (namn, objektnummer, adress, metadata)
// POST /api/v1/objects/bulk-action	– Massåtgärder (arkivera, flytta, ta bort)

Request/Response-format:

```
// POST /api/v1/objects – Skapa objekt
interface CreateObjectRequest {
  name: string;
  objectNumber: string;
  objectType: 'koncern' | 'brf' | 'fastighet' | 'rum' | 'karl' | 'custom';
  parentId?: string; // UUID av förälderobjekt
  streetAddress?: string;
  postalCode?: string;
  city?: string;
  latitude?: string;
  longitude?: string;
  vignetteImageUrl?: string;
  metadata?: Array<{
    fieldDefinitionId: string;
    value: string | number | null;
  }>;
}

// GET /api/v1/objects/:id – Response
interface ObjectDetailResponse {
  id: string;
  tenantId: string;
  name: string;
  objectNumber: string;
  objectType: string;
  status: string;
  parentId: string | null;
  parent?: { id: string; name: string; objectNumber: string; objectType: string };
  address: {
    streetAddress: string | null;
    postalCode: string | null;
    city: string | null;
    latitude: string | null;
    longitude: string | null;
  };
  vignetteImageUrl: string | null;
  childrenCount: number;
  metadataCount: number;
  taskCount: number;
  createdAt: string;
  updatedAt: string;
}

// GET /api/v1/objects/tree – Response
interface ObjectTreeResponse {
  tree: ObjectTreeNode[];
  summary: {
    koncern: number;
    brf: number;
    fastighet: number;
    rum: number;
    karl: number;
    total: number;
  };
}

interface ObjectTreeNode {
  id: string;
  name: string;
  objectNumber: string;
  objectType: string;
  childrenCount: number;
}
```



```

children?: ObjectTreeNode[];    // Laddas lazy vid expand
aggregatedKpis?: {
  totalChildren: number;        // Rekursivt
  totalKarl: number;
  monthlyValue: number;
};
}

```

Rekursiv CTE för trädtraversering (PostgreSQL):

```

-- Hämta hela subträdet under ett objekt
WITH RECURSIVE object_tree AS (
  -- Bas: startobjektet
  SELECT id, name, object_number, object_type, parent_id, 0 AS depth
  FROM objects
  WHERE id = $1 AND tenant_id = $2

  UNION ALL

  -- Rekursiv: alla barn
  SELECT o.id, o.name, o.object_number, o.object_type, o.parent_id, ot.depth + 1
  FROM objects o
  INNER JOIN object_tree ot ON o.parent_id = ot.id
  WHERE o.status = 'active'
)
SELECT * FROM object_tree ORDER BY depth, name;

```

UI-komponenter

Ny screen: ObjectTreeScreen

- Kollapsbar trädvy med ikoner per objekttyp
- Sammanfattningskort högst upp (antal per typ)
- Sökfält med filtrering
- Tap för att expandera/kollapsa noder
- Long-press för kontextmeny (redigera, flytta, ta bort)

Ny screen: ObjectDetailScreen

- Flikbaserad detaljvy (Översikt, Plats & Karta, Hierarki, Metadata, etc.)
- Redigeringsläge för grundinformation
- Bilduppladdning för vinjetbild

Ny screen: ObjectListScreen

- Tabellvy med sorterbara kolumner
- Anpassningsbara kolumner (välj vilka metadatafält som visas)
- Multiselekt med kryssrutor
- Sökfält med avancerad filtrering

3.2 Dynamiska metadatafält

Prioritet: ● Hög (Fas 1)

Affärskrav

- Varje objekt kan ha valfritt antal metadatafält
- Fältyper: alfanumerisk, numerisk, bild, fil, datum, boolean, dropdown

- Fältdefinitioner konfigureras per tenant (eller per objekttyp)
- Metadatavärden är sökbara och kan visas som kolumner i listvyer
- Metadata kan ärvas uppåt/nedåt i hierarkin

Drizzle-schema

```
export const metadataFieldTypeEnum = pgEnum('metadata_field_type', [
  'text', 'number', 'image', 'file', 'date', 'boolean', 'dropdown', 'multiselect'
]);

// Fältdefinitioner – mall för vilka fält som finns
export const metadataFieldDefinitions = pgTable('metadata_field_definitions', {
  id: uuid('id').defaultRandom().primaryKey(),
  tenantId: varchar('tenant_id', { length: 100 }).notNull(),
  name: varchar('name', { length: 200 }).notNull(),
  fieldKey: varchar('field_key', { length: 100 }).notNull(), // Maskin-läsbar
  nyckel
  fieldType: metadataFieldTypeEnum('field_type').notNull(),
  description: text('description'),
  isRequired: boolean('is_required').default(false),
  defaultValue: text('default_value'),
  dropdownOptions: jsonb('dropdown_options'), // För dropdown/
  multiselect: ["Val 1", "Val 2"]
  applicableObjectTypes: jsonb('applicable_object_types'), // ["rum", "karl"]
  eller null = alla
  sortOrder: integer('sort_order').default(0),
  isSearchable: boolean('is_searchable').default(true),
  isVisibleInList: boolean('is_visible_in_list').default(false), // Visa som kolumn i
  listvy
  createdAt: timestamp('created_at').defaultNow().notNull(),
}, (table) => [
  index('idx_mfd_tenant').on(table.tenantId),
  index('idx_mfd_tenant_key').on(table.tenantId, table.fieldKey),
]);

// Metadatavärden – per objekt
export const metadataValues = pgTable('metadata_values', {
  id: uuid('id').defaultRandom().primaryKey(),
  objectId: uuid('object_id').notNull().references(() => objects.id, { onDelete: 'cas-
  cade' }),
  fieldDefinitionId: uuid('field_definition_id').notNull()
    .references(() => metadataFieldDefinitions.id, { onDelete: 'cascade' }),
  textValue: text('text_value'),
  numberValue: doublePrecision('number_value'),
  booleanValue: boolean('boolean_value'),
  dateValue: timestamp('date_value'),
  fileUrl: text('file_url'),
  updatedAt: timestamp('updated_at').defaultNow().notNull(),
  updatedBy: varchar('updated_by', { length: 100 }),
}, (table) => [
  index('idx_mv_object').on(table.objectId),
  index('idx_mv_field_def').on(table.fieldDefinitionId),
  index('idx_mv_object_field').on(table.objectId, table.fieldDefinitionId),
  // Snabbsök på textvärden
  index('idx_mv_text_search').on(table.textValue),
]);
```

API-endpoints

GET	/api/v1/metadata/definitions	– Lista alla fältdefinitioner för tenant
POST	/api/v1/metadata/definitions	– Skapa ny fältdefinition
PUT	/api/v1/metadata/definitions/:id	– Uppdatera fältdefinition
DELETE	/api/v1/metadata/definitions/:id	– Ta bort fältdefinition
GET	/api/v1/objects/:id/metadata	– Hämta alla metadatavärden för ett objekt
PUT	/api/v1/objects/:id/metadata	– Uppdatera metadatavärden (batch)
PUT	/api/v1/objects/:id/metadata/:fieldId	– Uppdatera enskilt metadatavärde

UI-komponent: `DynamicMetadataForm`

```
// Renderar ett formulär baserat på fältdefinitioner
interface DynamicMetadataFormProps {
  objectId: string;
  definitions: MetadataFieldDefinition[];
  values: MetadataValue[];
  onSave: (values: MetadataValue[]) => void;
  readOnly?: boolean;
}

function DynamicMetadataForm({ definitions, values, onSave, readOnly }: Dynamic-
MetadataFormProps) {
  return (
    <ScrollView>
      {definitions.map(def => {
        const currentValue = values.find(v => v.fieldDefinitionId === def.id);
        switch (def.fieldType) {
          case 'text':
            return <TextInput key={def.id} label={def.name} value={currentValue?.text-
Value} />;
          case 'number':
            return <NumberInput key={def.id} label={def.name} value={currentValue?.num-
berValue} />;
          case 'dropdown':
            return <DropdownPicker key={def.id} label={def.name} options={def.dropdown
Options} />;
          case 'image':
            return <ImagePicker key={def.id} label={def.name} uri={currentValue?.fileU-
rl} />;
          case 'date':
            return <DatePicker key={def.id} label={def.name} value={currentValue?.date
Value} />;
          case 'boolean':
            return <Switch key={def.id} label={def.name} value={currentValue?.boolean-
Value} />;
          // ... fler typer
        }
      })}
    </ScrollView>
  );
}
```

3.3 Orderkoncept och uppgiftsmodell

Prioritet: 🟡 Medium (Fas 2)

Affärskrav

- **Orderkoncept** representerar ett abonnemang, avrop eller schema
- Ett orderkoncept kan kopplas till **flera objekt** (many-to-many)
- Orderkoncept genererar **uppgifter** med leveranstider
- **Felanmälningar** blir en typ av uppgift
- Dubbelriktad navigering: Objekt ↔ Uppgifter ↔ Orderkoncept

Drizzle-schema

```

export const orderConceptTypeEnum = pgEnum('order_concept_type', [
  'subscription', 'calloff', 'schedule', 'one_time'
]);

export const taskStatusEnum = pgEnum('task_status', [
  'planned', 'active', 'in_progress', 'completed', 'invoiced', 'cancelled'
]);

export const taskTypeEnum = pgEnum('task_type', [
  'service', 'fault_report', 'inspection', 'delivery', 'pickup', 'custom'
]);

export const orderConcepts = pgTable('order_concepts', {
  id: uuid('id').defaultRandom().primaryKey(),
  tenantId: varchar('tenant_id', { length: 100 }).notNull(),
  name: varchar('name', { length: 500 }).notNull(),
  type: orderConceptTypeEnum('type').notNull(),
  description: text('description'),
  orderReference: varchar('order_reference', { length: 200 }), // Beställningsreferens
  customerReference: varchar('customer_reference', { length: 200 }),

  // Schema/Abonnemang
  startDate: timestamp('start_date'),
  endDate: timestamp('end_date'),
  recurrenceRule: text('recurrence_rule'), // iCal RRULE-format

  status: varchar('status', { length: 50 }).default('active'),
  createdAt: timestamp('created_at').defaultNow().notNull(),
  updatedAt: timestamp('updated_at').defaultNow().notNull(),
}, (table) => [
  index('idx_oc_tenant').on(table.tenantId),
  index('idx_oc_type').on(table.type),
]);

// Many-to-many: orderkoncept ↔ objekt
export const orderConceptObjects = pgTable('order_concept_objects', {
  id: uuid('id').defaultRandom().primaryKey(),
  orderConceptId: uuid('order_concept_id').notNull()
    .references(() => orderConcepts.id, { onDelete: 'cascade' }),
  objectId: uuid('object_id').notNull()
    .references(() => objects.id, { onDelete: 'cascade' }),
}, (table) => [
  index('idx_oco_concept').on(table.orderConceptId),
  index('idx_oco_object').on(table.objectId),
]);

export const tasks = pgTable('tasks', {
  id: uuid('id').defaultRandom().primaryKey(),
  tenantId: varchar('tenant_id', { length: 100 }).notNull(),
  objectId: uuid('object_id').references(() => objects.id, { onDelete: 'set null' }),
  orderConceptId: uuid('order_concept_id')
    .references(() => orderConcepts.id, { onDelete: 'set null' }),

  type: taskTypeEnum('type').notNull().default('service'),
  description: text('description'),
  status: taskStatusEnum('status').notNull().default('planned'),

  // Tidsramar
  deliveryWindowStart: timestamp('delivery_window_start'),
  deliveryWindowEnd: timestamp('delivery_window_end'),
  scheduledDate: timestamp('scheduled_date'),

```

```

estimatedDuration: integer('estimated_duration_minutes'),

// Tilldelning
assignedUserId: varchar('assigned_user_id', { length: 100 }),
assignedTeamId: varchar('assigned_team_id', { length: 100 }),

// Utförande
startedAt: timestamp('started_at'),
completedAt: timestamp('completed_at'),
actualDuration: integer('actual_duration_minutes'),

// Felanmälan-specifikt
faultDescription: text('fault_description'),
faultSeverity: varchar('fault_severity', { length: 50 }),
faultReportedBy: varchar('fault_reported_by', { length: 200 }),
faultReportedAt: timestamp('fault_reported_at'),

createdAt: timestamp('created_at').defaultNow().notNull(),
updatedAt: timestamp('updated_at').defaultNow().notNull(),
}, (table) => [
  index('idx_tasks_tenant').on(table.tenantId),
  index('idx_tasks_object').on(table.objectId),
  index('idx_tasks_concept').on(table.orderConceptId),
  index('idx_tasks_status').on(table.status),
  index('idx_tasks_scheduled').on(table.scheduledDate),
  index('idx_tasks_assigned').on(table.assignedUserId),
]);

```

API-endpoints

# Orderkoncept	
GET /api/v1/order-concepts	Lista orderkoncept
POST /api/v1/order-concepts	Skapa orderkoncept
GET /api/v1/order-concepts/:id	Hämta orderkoncept med objekt & uppgifter
PUT /api/v1/order-concepts/:id	Uppdatera
DELETE /api/v1/order-concepts/:id	Ta bort
POST /api/v1/order-concepts/:id/objects	Koppla objekt till orderkoncept
DELETE /api/v1/order-concepts/:id/objects/:objectId	Ta bort objektkoppling
POST /api/v1/order-concepts/:id/generate-tasks	Generera uppgifter från schema
# Uppgifter	
GET /api/v1/tasks	Lista uppgifter (filtrering, paginering)
POST /api/v1/tasks	Skapa uppgift
GET /api/v1/tasks/:id	Hämta uppgift
PUT /api/v1/tasks/:id	Uppdatera
PATCH /api/v1/tasks/:id/status	Ändra status
GET /api/v1/objects/:id/tasks	Uppgifter för ett objekt
GET /api/v1/tasks/calendar	Kalendervy-data (grupperat per dag/vecka)
# Felanmälningar (specialfall av uppgifter)	
POST /api/v1/fault-reports	Skapa felanmälan (→ uppgift med type=fault_report)
GET /api/v1/objects/:id/fault-reports	Felanmälningar per objekt

3.4 Kunddashboard

Prioritet: 🟡 Medium (Fas 2)

Affärskrav

- Översiktssida per kund/toppnivåobjekt med KPI-kort
- Visa: Antal objekt, aktiva uppgifter, månadsvärde, snitt ställtid
- "Räkna om arv" – trigga kaskadberäkning av ärvda värden

API-endpoints

GET /api/v1/dashboard/:tenantId/summary

Response:

```
{
  "customer": { "name": "Södertälje Kommun", "id": "..." },
  "kpis": {
    "totalObjects": 891,
    "activeTaskCount": 12,
    "monthlyValue": 45000,           // kr
    "avgSetupTime": 15,             // minuter
    "completedThisMonth": 34,
    "invoicedThisMonth": 28
  },
  "objectTypeCounts": {
    "koncern": 1, "brf": 1, "fastighet": 10, "rum": 220, "karl": 659
  },
  "pipeline": {
    "objects": 891,
    "subscriptions": 5,
    "activeTasks": 12,
    "completed": 34,
    "invoiced": 28
  },
  "monthlyTrend": [
    { "month": "2026-01", "value": 42000, "tasks": 30 },
    { "month": "2026-02", "value": 44000, "tasks": 35 },
    // ...
  ]
}
```

POST /api/v1/dashboard/:tenantId/recalculate-inheritance

// Triggas bakgrundsjobb som beräknar om ärvda KPI:er genom hela hierarkin

UI-komponent: CustomerDashboard

```
function CustomerDashboard({ tenantId }: { tenantId: string }) {
  const { data } = useQuery(['dashboard', tenantId], () => fetchDashboard(tenantId));

  return (
    <View>
      {/* KPI-kort rad */}
      <View style={styles.kpiRow}>
        <KpiCard icon="package" label="Objekt" value={data.kpis.totalObjects} />
        <KpiCard icon="clipboard" label="Aktiva uppgifter" value={data.kpis.activeTask
Count} />
        <KpiCard icon="dollar" label="Månadsvärde" value={`$${data.kpis.monthlyValue}
kr`} />
        <KpiCard icon="clock" label="Snitt ställtid" value={`$
{data.kpis.avgSetupTime} min`} />
      </View>

      {/* Objekttyp-sammanfattning */}
      <ObjectTypeSummary counts={data.objectTypeCounts} />

      {/* Pipeline-visualisering */}
      <PipelineFlow steps={data.pipeline} />

      {/* Månadsdiagram */}
      <MonthlyTrendChart data={data.monthlyTrend} />
    </View>
  );
}
```

3.5 Kalendervy / Tidsperspektiv**Prioritet:** ● Låg (Fas 3)**Affärskrav**

- Expanderbar kalendervy: År → Kvartal → Månad → Vecka → Dag
- Visa uppgifter kopplade till objekt med leveranstidsfönster
- Kunden ska kunna se "när kommer ni?" i kalendern

API-endpoints

```
GET /api/v1/calendar
Query params:
- startDate: ISO date
- endDate: ISO date
- objectId?: UUID (filtrera per objekt)
- assignedUserId?: UUID
- view: 'year' | 'quarter' | 'month' | 'week' | 'day'

Response:
{
  "view": "month",
  "period": { "start": "2026-06-01", "end": "2026-06-30" },
  "days": [
    {
      "date": "2026-06-10",
      "taskCount": 3,
      "tasks": [
        {
          "id": "...",
          "description": "Kärltömning Hemköp Norr",
          "objectName": "Norr Handlarägd",
          "deliveryWindowStart": "08:00",
          "deliveryWindowEnd": "12:00",
          "status": "planned",
          "assignedUser": "Erik Karlsson"
        }
      ]
    }
  ]
}
```

UI-komponent: `CalendarView`

- Använd `react-native-calendars` eller egen implementation
- Zoombar: pinch-to-zoom byter granularitet
- Färgkodade events baserat på status
- Drag-and-drop för omschemaläggning (webb)

3.6 Kundportal

Prioritet: ● Låg (Fas 3)

Affärskrav

- Kundanvändare (roll = `customer`) ser **enbart sina objekt** i hierarkin
- Kan navigera sin butik, se kontaktpersoner, metadata, barn-objekt
- Kan se kopplade uppgifter med tidsperspektiv
- Får push-notiser för tilldelade uppgifter
- Kan skapa felanmälningar

Teknisk lösning

Ny koppling: användare ↔ rotnivå-objekt

```
export const userObjectAccess = pgTable('user_object_access', {
  id: uuid('id').defaultRandom().primaryKey(),
  userId: varchar('user_id', { length: 100 }).notNull(),
  objectId: uuid('object_id').notNull()
    .references(() => objects.id, { onDelete: 'cascade' }),
  accessLevel: varchar('access_level', { length: 50 }).default('read'), // read,
  write, admin
  createdAt: timestamp('created_at').defaultNow().notNull(),
}, (table) => [
  index('idx_uoa_user').on(table.userId),
  index('idx_uoa_object').on(table.objectId),
]);
```

Middleware: requireCustomerAuth

```
async function requireCustomerAuth(req, res, next) {
  // Verifiera token
  const userId = res.locals.mobileResourceId;
  const userRole = res.locals.role;

  if (userRole === 'customer') {
    // Hämta tillåtna rot-objekt
    const accessibleObjects = await db.select()
      .from(userObjectAccess)
      .where(eq(userObjectAccess.userId, userId));

    // Sätt på request för filtrering nedströms
    res.locals.accessibleObjectIds = accessibleObjects.map(a => a.objectId);
    res.locals.isCustomerPortal = true;
  }

  next();
}
```

Filtrering i objekt-queries:

- Alla objekt-endpoints kontrollerar `res.locals.isCustomerPortal`
 - Om `true` : filtrera till objekt som är barn (rekursivt) av `accessibleObjectIds`
 - Använd CTE för att hitta alla tillåtna objekt i subträdet
-

4. DATAMODELL-UTÖKNINGAR

4.1 Nya tabeller

Tabell	Syfte	Uppskattade rader
objects	Kärn-entitet – alla objekt i hierarkin	10 000+
metadata_field_definitions	Fältdefinitioner per tenant	50–200 per tenant
metadata_values	Metadatatvärden per objekt	50 000+
order_concepts	Abonnemang, avrop, schema	100–1 000
order_concept_objects	Many-to-many orderkoncept ↔ objekt	1 000–10 000
tasks	Uppgifter (arbetsordrar, felanmälningar)	10 000+
user_object_access	Kundportal-åtkomst	100–1 000
contacts	Kontaktpersoner per objekt	1 000–5 000
activity_log	Ändringshistorik per objekt	100 000+

4.2 Ändringar i befintliga tabeller

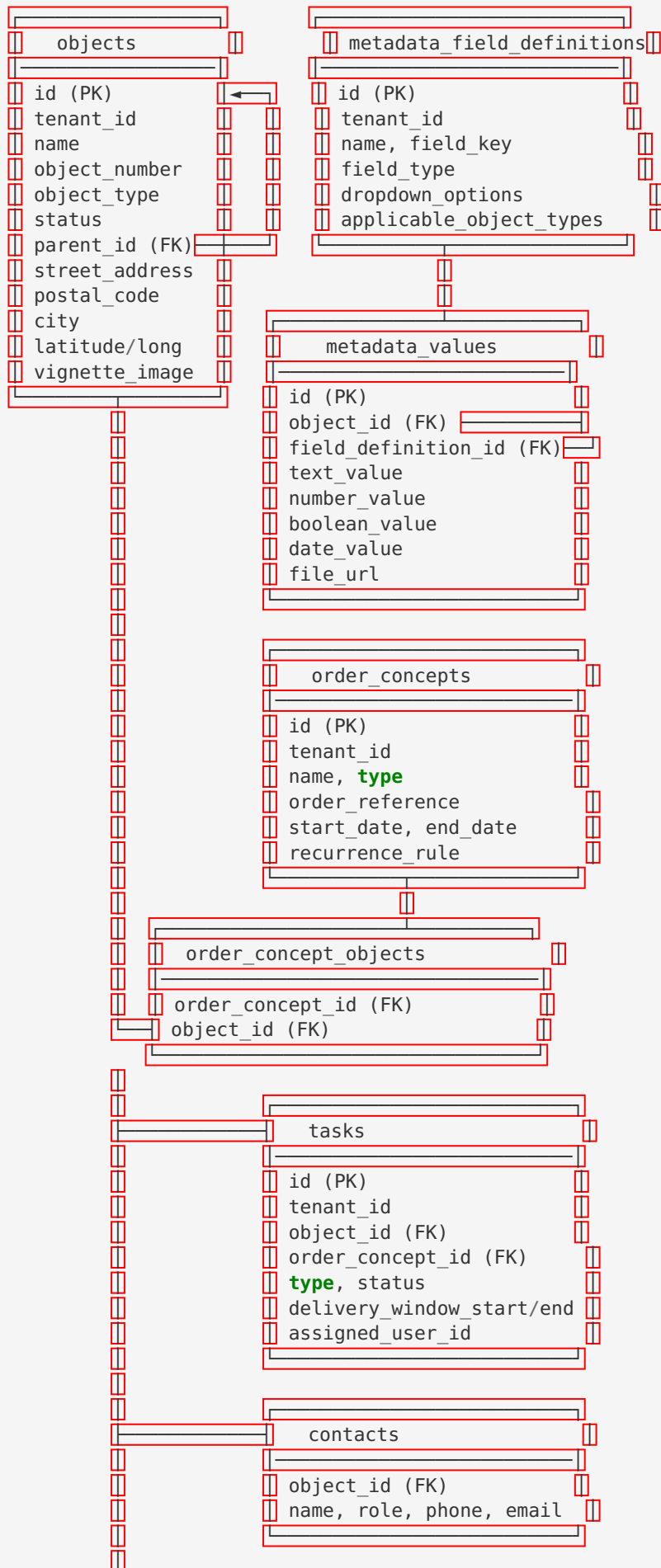
Tabell	Ändring
time_entries	Lägg till task_id (FK → tasks) utöver order_id
inspection_photos	Lägg till object_id (FK → objects) utöver order_id

4.3 Nya tabeller - fullständiga definitioner

```
// Kontaktpersoner per objekt
export const contacts = pgTable('contacts', {
  id: uuid('id').defaultRandom().primaryKey(),
  objectId: uuid('object_id').notNull()
    .references(() => objects.id, { onDelete: 'cascade' }),
  name: varchar('name', { length: 300 }).notNull(),
  role: varchar('role', { length: 100 }), // "Butikschef", "Fastighetsans-
    varig"
  phone: varchar('phone', { length: 50 }),
  email: varchar('email', { length: 200 }),
  isPrimary: boolean('is_primary').default(false),
  createdAt: timestamp('created_at').defaultNow().notNull(),
}, (table) => [
  index('idx_contacts_object').on(table.objectId),
]);

// Ändringshistorik
export const activityLog = pgTable('activity_log', {
  id: uuid('id').defaultRandom().primaryKey(),
  tenantId: varchar('tenant_id', { length: 100 }).notNull(),
  entityType: varchar('entity_type', { length: 50 }).notNull(), // 'object', 'task',
    'order_concept'
  entityId: uuid('entity_id').notNull(),
  action: varchar('action', { length: 50 }).notNull(), // 'created', 'up-
    dated', 'moved', 'archived'
  changes: jsonb('changes'), // { field:
    { old: ..., new: ... } }
  performedBy: varchar('performed_by', { length: 100 }),
  performedAt: timestamp('performed_at').defaultNow().notNull(),
}, (table) => [
  index('idx_al_entity').on(table.entityType, table.entityId),
  index('idx_al_tenant').on(table.tenantId),
  index('idx_al_time').on(table.performedAt),
]);
```

4.4 Relationsdiagram (ER)





5. API-SPECIFIKATION

5.1 Nya endpoints - sammanfattning

Metod	Endpoint	Beskrivning	Prio
GET	/api/v1/objects	Lista objekt (paginering, filtrering, sökning)	Hög
POST	/api/v1/objects	Skapa objekt	Hög
GET	/api/v1/objects/:id	Hämta objekt med relationer	Hög
PUT	/api/v1/objects/:id	Uppdatera objekt	Hög
DELETE	/api/v1/objects/:id	Soft-delete (arkivera)	Hög
GET	/api/v1/objects/:id/children	Barnobjekt	Hög
GET	/api/v1/objects/:id/ancestors	Förälderkedja (breadcrumb)	Hög
PATCH	/api/v1/objects/:id/move	Flytta till ny förälder	Hög
GET	/api/v1/objects/tree	Hämta hierarkiträd	Hög
GET	/api/v1/objects/search	Fritextsökning	Hög
POST	/api/v1/objects/bulk-action	Massåtgärder	Medium
GET	/api/v1/metadata/definitions	Lista fältdefinitioner	Hög
POST	/api/v1/metadata/definitions	Skapa fältdefinition	Hög
PUT	/api/v1/metadata/definitions/:id	Uppdatera fältdefinition	Medium
DELETE	/api/v1/metadata/definitions/:id	Ta bort fältdefinition	Medium
GET	/api/v1/objects/:id/metadata	Hämta objekt-metadata	Hög
PUT			Hög

Metod	Endpoint	Beskrivning	Prio
	/api/v1/objects/:id/ metadata	Uppdatera metadata (batch)	
GET	/api/v1/order-con- cepts	Lista orderkoncept	Medium
POST	/api/v1/order-con- cepts	Skapa orderkoncept	Medium
GET	/api/v1/order-con- cepts/:id	Hämta orderkoncept	Medium
PUT	/api/v1/order-con- cepts/:id	Uppdatera	Medium
DELETE	/api/v1/order-con- cepts/:id	Ta bort	Medium
POST	/api/v1/order-con- cepts/:id/objects	Koppla objekt	Medium
POST	/api/v1/order-con- cepts/:id/generate- tasks	Generera uppgifter	Medium
GET	/api/v1/tasks	Lista uppgifter	Medium
POST	/api/v1/tasks	Skapa uppgift	Medium
GET	/api/v1/tasks/:id	Hämta uppgift	Medium
PUT	/api/v1/tasks/:id	Uppdatera	Medium
PATCH	/api/v1/tasks/:id/ status	Ändra status	Medium
GET	/api/v1/objects/:id/ tasks	Uppgifter per objekt	Medium
GET	/api/v1/tasks/calen- dar	Kalendervy-data	Låg
POST	/api/v1/fault-re- ports	Skapa felanmälan	Medium
GET	/api/v1/dash- board/:tenantId/sum- mary	Kunddashboard	Medium

Metod	Endpoint	Beskrivning	Prio
POST	/api/v1/dashboard/:tenantId/re-calculate	Omberäkna arv	Medium
GET	/api/v1/calendar	Kalendervy	Låg

5.2 Autentisering och behörigheter

Alla nya endpoints skyddas av befintlig `requireMobileAuth` -middleware. Tillägg:

```
// Ny middleware: rollbaserad åtkomst
function requireRole(...allowedRoles: string[]) {
  return (req, res, next) => {
    const role = res.locals.role || 'user';
    if (!allowedRoles.includes(role)) {
      return res.status(403).json({ error: 'Otillräcklig behörighet' });
    }
    next();
  };
}

// Ny middleware: kundportal-filtrering
function applyCustomerFilter(req, res, next) {
  if (res.locals.isCustomerPortal) {
    // Filtrera queries till enbart tillåtna objekt
    req.accessibleObjectIds = res.locals.accessibleObjectIds;
  }
  next();
}

// Användning:
router.get('/objects', requireMobileAuth, applyCustomerFilter, objectController.list);
router.post('/objects', requireMobileAuth, requireRole('admin', 'planner'), objectController.create);
router.delete('/objects/:id', requireMobileAuth, requireRole('admin'), objectController.delete);
```

5.3 Paginering och filtrering (standard)

Alla list-endpoints följer samma mönster:

```
// Query-parametrar (GET /api/v1/objects)
interface ListQueryParams {
  page?: number;           // Default: 1
  pageSize?: number;       // Default: 50, Max: 200
  sort?: string;           // Fältnamn, prefix med - för DESC: "-name"
  search?: string;         // Fritextsökning
  status?: string;         // Filter: "active", "archived"
  objectType?: string;     // Filter: "koncern", "brf", etc.
  parentId?: string;       // Filter: direkta barn av denna förälder
  tenantId?: string;       // Automatiskt från auth
}

// Standard-response
interface PaginatedResponse<T> {
  data: T[];
  pagination: {
    page: number;
    pageSize: number;
    totalItems: number;
    totalPages: number;
    hasMore: boolean;
  };
}
```

6. FRONTEND-KOMPONENTER

6.1 Nya screens

Screen	Beskrivning	Plattform	Prio
ObjectTreeScreen	Hierarkisk trädvy med drill-down	Mobil + Webb	Hög
ObjectListScreen	Tabellvy med anpassningsbara kolumner	Webb (primärt)	Hög
ObjectDetailScreen	Flikbaserad detaljvy (13+ flikar)	Mobil + Webb	Hög
ObjectCreateEditScreen	Formulär för skapa/redigera objekt	Mobil + Webb	Hög
MetadataAdminScreen	Admin: hantera fältdefinitioner	Webb	Hög
CustomerDashboardScreen	KPI-kort + processflöde + diagram	Webb	Medium
OrderConceptScreen	Lista/detalj orderkoncept	Webb	Medium
TaskListScreen	Uppgiftslista med filtrering	Mobil + Webb	Medium
CalendarScreen	Expanderbar kalender	Webb	Låg
FaultReportScreen	Skapa felanmälan	Mobil + Webb	Medium
Customer-PortalScreen	Filtrerad hierarki för kunder	Webb	Låg

6.2 Nya återanvändbara komponenter

components/	
├─ objects/	
│ └─ ObjectTreeView.tsx	# Kollapsbar trädvy med ikoner
│ └─ ObjectTreeNode.tsx	# Enskild nod i trädet
│ └─ ObjectCard.tsx	# Kort för objekt i lista/grid
│ └─ ObjectBreadcrumb.tsx	# Breadcrumb-navigering
│ └─ ObjectTypeBadge.tsx	# Ikon + etikett per objekttyp
│ └─ ObjectTypeIcon.tsx	# Ikon per typ (🏠🏢🏡🏠🏡)
│ └─ ObjectSearchBar.tsx	# Sökfält med avancerad filtrering
│ └─ ObjectSummaryCards.tsx	# Sammanfattningskort (antal per typ)
│ └─ ObjectMoveModal.tsx	# Modal för att flytta objekt
├─ metadata/	
│ └─ DynamicMetadataForm.tsx	# Renderare för dynamiska fält
│ └─ MetadataFieldRenderer.tsx	# Renderar enskilt fält baserat på typ
│ └─ MetadataFieldEditor.tsx	# Admin: redigera fältdefinition
│ └─ ColumnSelector.tsx	# Välj vilka fält som visas i listvy
├─ dashboard/	
│ └─ KpiCard.tsx	# Enskilt KPI-kort med ikon + värde
│ └─ PipelineFlow.tsx	# Processflödesvisualisering
│ └─ MonthlyTrendChart.tsx	# Årsdiagram (Recharts/Victory)
│ └─ ObjectTypeSummary.tsx	# Sammanfattning per objekttyp
├─ tasks/	
│ └─ TaskCard.tsx	# Uppgiftskort
│ └─ TaskStatusBadge.tsx	# Statusindikator
│ └─ TaskTimeline.tsx	# Tidslinje för uppgifter
├─ calendar/	
│ └─ CalendarView.tsx	# Huvudkomponent
│ └─ CalendarDay.tsx	# Dagsvy
│ └─ CalendarMonth.tsx	# Månadsvy
│ └─ CalendarEvent.tsx	# Enskild event/uppgift
├─ shared/	
│ └─ DataTable.tsx	# Generisk tabell med sort/filter/paginerings
│ └─ TreeView.tsx	# Generisk trädvy
│ └─ SearchWithFilters.tsx	# Sökfält + filterchips
│ └─ BulkActionBar.tsx	# Toolbar för massåtgärder
│ └─ TabPanel.tsx	# Flikbaserad container
│ └─ EmptyState.tsx	# Placeholder vid tom data
│ └─ InfiniteScrollList.tsx	# Oändlig scroll-lista

6.3 Navigeringsändringar

Nya flikar i `TabNavigator` :

```
// Förslag: Utöka befintliga flikar med ny "Objekt"-flik
const tabs = [
  { name: 'home', icon: 'home', label: 'Hem' },
  { name: 'objects', icon: 'layers', label: 'Objekt' }, // NY
  { name: 'orders', icon: 'clipboard', label: 'Uppgifter' }, // Byt namn från
  "Ordrrar"
  { name: 'map', icon: 'map', label: 'Karta' },
  { name: 'more', icon: 'menu', label: 'Mer' },
];
```

Nya stack-screens i RootNavigator :

```
// Lägg till i RootNavigator
<Stack.Screen name="ObjectTree" component={ObjectTreeScreen} />
<Stack.Screen name="ObjectDetail" component={ObjectDetailScreen} />
<Stack.Screen name="ObjectCreateEdit" component={ObjectCreateEditScreen} />
<Stack.Screen name="ObjectList" component={ObjectListScreen} />
<Stack.Screen name="MetadataAdmin" component={MetadataAdminScreen} />
<Stack.Screen name="CustomerDashboard" component={CustomerDashboardScreen} />
<Stack.Screen name="OrderConcept" component={OrderConceptScreen} />
<Stack.Screen name="TaskList" component={TaskListScreen} />
<Stack.Screen name="TaskDetail" component={TaskDetailScreen} />
<Stack.Screen name="Calendar" component={CalendarScreen} />
<Stack.Screen name="FaultReport" component={FaultReportScreen} />
```

6.4 State management

Befintlig arkitektur med **React Query** + **Context** behålls. Tillägg:

```
// Nya query-nycklar
const QUERY_KEYS = {
  objects: {
    all: ['objects'] as const,
    lists: () => [...QUERY_KEYS.objects.all, 'list'] as const,
    list: (filters: ObjectFilters) => [...QUERY_KEYS.objects.lists(), filters] as const,
  },
  tree: (parentId?: string) => [...QUERY_KEYS.objects.all, 'tree', parentId] as const,
  details: () => [...QUERY_KEYS.objects.all, 'detail'] as const,
  detail: (id: string) => [...QUERY_KEYS.objects.details(), id] as const,
  children: (id: string) => [...QUERY_KEYS.objects.all, 'children', id] as const,
  ancestors: (id: string) => [...QUERY_KEYS.objects.all, 'ancestors', id] as const,
  search: (query: string) => [...QUERY_KEYS.objects.all, 'search', query] as const,
},
  metadata: {
    definitions: (tenantId: string) => ['metadata', 'definitions', tenantId] as const,
    values: (objectId: string) => ['metadata', 'values', objectId] as const,
  },
  tasks: {
    all: ['tasks'] as const,
    list: (filters: TaskFilters) => ['tasks', 'list', filters] as const,
    detail: (id: string) => ['tasks', 'detail', id] as const,
    calendar: (params: CalendarParams) => ['tasks', 'calendar', params] as const,
    byObject: (objectId: string) => ['tasks', 'byObject', objectId] as const,
  },
  dashboard: {
    summary: (tenantId: string) => ['dashboard', 'summary', tenantId] as const,
  },
  orderConcepts: {
    all: ['orderConcepts'] as const,
    detail: (id: string) => ['orderConcepts', id] as const,
  },
};

// Ny context: ObjectNavigationContext (för breadcrumb + trädstate)
interface ObjectNavigationState {
  currentObjectId: string | null;
  expandedNodes: Set<string>;
  breadcrumb: Array<{ id: string; name: string; objectType: string }>;
}
```


7. INTEGRATION OCH MIGRATION

7.1 Integrationsstrategi

De nya funktionerna integreras **progressivt** utan att bryta befintlig funktionalitet:

Steg 1: Databasutökning

1. Skapa nya tabeller med `ensure-tables` -mönstret (CREATE TABLE IF NOT EXISTS)
2. Behåll alla befintliga 6 tabeller oförändrade initialt
3. Lägg till nya tabellscheman i `lib/db/src/schema/` som separata filer

```
lib/db/src/schema/
├── traivo.ts           # Befintliga 6 tabeller (ÄNDRA INTE)
├── objects.ts         # NY: objects + contacts
├── metadata.ts        # NY: metadata_field_definitions + metadata_values
├── orderConcepts.ts   # NY: order_concepts + order_concept_objects
├── tasks.ts           # NY: tasks
├── access.ts          # NY: user_object_access
├── activityLog.ts     # NY: activity_log
└── index.ts           # Exportera alla
```

Steg 2: API-utökning

1. Skapa nya route-filer under `artifacts/api-server/src/routes/`
2. Montera nya rutter i `routes/index.ts` under `/api/v1/`
3. Behåll alla befintliga rutter intakta

```
// routes/index.ts – tillägg
import { objectsRouter } from './objects';
import { metadataRouter } from './metadata';
import { tasksRouter } from './tasks';
import { orderConceptsRouter } from './orderConcepts';
import { dashboardRouter } from './dashboard';
import { calendarRouter } from './calendar';

// Under befintliga monteringar:
router.use('/v1/objects', requireMobileAuth, objectsRouter);
router.use('/v1/metadata', requireMobileAuth, metadataRouter);
router.use('/v1/tasks', requireMobileAuth, tasksRouter);
router.use('/v1/order-concepts', requireMobileAuth, orderConceptsRouter);
router.use('/v1/dashboard', requireMobileAuth, dashboardRouter);
router.use('/v1/calendar', requireMobileAuth, calendarRouter);
```

Steg 3: Frontend-tillägg

1. Lägg till nya screens i `screens/`
2. Lägg till nya komponenter i `components/`
3. Utöka navigeringen med nya stack-screens
4. **Ändra INTE** befintliga screens initialt

7.2 Mock-data för nya entiteter

Utöka `mockData.ts` med objekt-, metadata-, och uppgiftsdata:

```
// I mockData.ts eller ny fil mockObjects.ts

export const MOCK_OBJECT_TYPES = [
  { key: 'koncern', label: 'Koncern', icon: '🏢' },
  { key: 'brf', label: 'BRF', icon: '🏠' },
  { key: 'fastighet', label: 'Fastighet', icon: '🏡' },
  { key: 'rum', label: 'Rum', icon: '🛏' },
  { key: 'karl', label: 'Kärl', icon: '🗑' },
];

export const MOCK_OBJECTS: ObjectTreeNode[] = [
  {
    id: 'obj-001',
    name: 'Axfood Sverige',
    objectNumber: 'AXF-001',
    objectType: 'koncern',
    status: 'active',
    parentId: null,
    children: [
      {
        id: 'obj-002',
        name: 'Hemköp kedjan',
        objectNumber: 'HEM-001',
        objectType: 'brf',
        parentId: 'obj-001',
        children: [
          {
            id: 'obj-003',
            name: 'Stockholm Egenägd',
            objectNumber: 'STH-E01',
            objectType: 'fastighet',
            parentId: 'obj-002',
            address: { streetAddress: 'Storgatan 12', postalCode: '111 23', city: 'Stockholm' },
            children: [
              { id: 'obj-010', name: 'Pantrum A', objectNumber: 'STH-E01-R01', objectType: 'rum', parentId: 'obj-003' },
              { id: 'obj-011', name: 'Kundvagnsgarage', objectNumber: 'STH-E01-R02', objectType: 'rum', parentId: 'obj-003' },
            ]
          },
          // ... fler fastigheter
        ]
      }
    ]
  }
];

export const MOCK_METADATA_DEFINITIONS = [
  { id: 'mfd-001', name: 'Antal kärl', fieldKey: 'antal_karl', fieldType: 'number', applicableObjectTypes: ['rum'] },
  { id: 'mfd-002', name: 'Kärlstorlek (liter)', fieldKey: 'karl_storlek', fieldType: 'dropdown', dropdownOptions: ['140', '240', '370', '660', '1100'] },
  { id: 'mfd-003', name: 'Tönningsfrekvens', fieldKey: 'tomningsfrekvens', fieldType: 'text' },
  { id: 'mfd-004', name: 'Åtkomstbeskrivning', fieldKey: 'atkomst', fieldType: 'text' },
  { id: 'mfd-005', name: 'Sista inspektion', fieldKey: 'sista_inspektion', fieldType: 'date' },
];
```

7.3 Bakåtkompatibilitet

- **Alla befintliga endpoints** (/api/v1/mobile/*) behålls oförändrade
- **Alla befintliga DB-tabeller** behålls oförändrade
- **Mobilappen** fortsätter fungera som förut – nya funktioner är additioner
- Befintlig `Order` -modell kan gradvis utökas med `objectId` - och `taskId` -referenser
- Mock/live-dualiteten respekteras i alla nya rutter

7.4 Testing-strategi

1. Unit tests (per route/**controller**):
 - Objekthantering CRUD
 - Hierarkitraversering (CTE-queries)
 - Metadata-**validering**
 - Behörighetskontroller
2. **Integration** tests:
 - Skapa objekthierarki ➡ navigera ➡ redigera ➡ flytta ➡ ta bort
 - Metadata: **definiera** fält ➡ sätta värden ➡ söka
 - **Order**koncept ➡ generera uppgifter ➡ statusövergångar
3. E2E tests (befintlig setup med mock):
 - Navigera trädvy ➡ drilla ner ➡ visa detalj
 - Skapa objekt med metadata ➡ verifiera i **lista**
 - Kundportal: **logga** in som kund ➡ se filtrerad vy

8. PRIORITERAD ROADMAP

Fas 1 - MVP (Kritiska funktioner)

Mål: Grundläggande objekthantering med hierarki och metadata.

#	Funktion	Beskrivning	Uppskattad effort
1.1	DB-schema: objects	Skapa <code>objects</code> -tabellen med self-referencing FK	2-4h
1.2	DB-schema: metadata	Skapa <code>metadata_field_definitions</code> + <code>metadata_values</code>	2-4h
1.3	Objekt CRUD API	Alla endpoints för skapa/läsa/uppdatera/ta bort	8-12h
1.4	Hierarki API	Tree, children, ancestors, move	6-8h
1.5	Metadata API	Definitions CRUD + values per objekt	6-8h
1.6	Mock-data	Realistisk objekthierarki + metadata	4-6h
1.7	ObjectTreeScreen	Trädvy med drill-down (mobil)	8-12h
1.8	ObjectDetailScreen	Flikbaserad detaljvy	8-12h
1.9	DynamicMetadata-Form	Renderare för dynamiska metadatafält	6-8h
1.10	Sök och filtrering	Fritextsökning i objektlista	4-6h

Estimat Fas 1: ~55-80 timmar

Fas 2 - Förbättring (Viktiga funktioner)

Mål: Webb-dashboard, orderkoncept, uppgiftshantering.

#	Funktion	Beskrivning	Uppskattad effort
2.1	DB-schema: tasks + order_concepts	Skapa tabeller	3-4h
2.2	Orderkoncept API	CRUD + objektkoppling	6-8h
2.3	Uppgifter API	CRUD + statusövergångar	6-8h
2.4	Felanmälningar	Skapa som uppgiftstyp	3-4h
2.5	Kunddashboard API	KPI-beräkningar + sammanfattning	6-8h
2.6	CustomerDashboard-Screen	KPI-kort + pipeline + diagram	8-12h
2.7	ObjectListScreen (webb)	Anpassningsbara kolumner i mockup-sandbox	8-12h
2.8	Kontakter API	CRUD per objekt	3-4h
2.9	Ändringshistorik	Activity log per objekt	4-6h
2.10	Dubbelriktad navigering	Objekt ↔ Uppgifter ↔ Orderkoncept	4-6h

Estimat Fas 2: ~50-70 timmar

Fas 3 - Optimering (Nice-to-have)

Mål: Kalender, kundportal, avancerade funktioner.

#	Funktion	Beskrivning	Uppskattad effort
3.1	Kalendervy API	Dataaggregering per dag/vecka/månad	6–8h
3.2	CalendarScreen	Expanderbar kalender (År→Dag)	12–16h
3.3	Kundportal	Roll-filtrering + user_object_access	8–12h
3.4	Push-notiser (kunder)	Aviseringar för tilldelade uppgifter	4–6h
3.5	Massåtgärder	Bulk arkivera/flytta/ta bort	4–6h
3.6	Metadata-arv	Kaskadberäkning genom hierarkin	6–8h
3.7	Telink-integration	Synka objektdata från externt system	8–12h
3.8	SlotPreference	Tidsfönsterpreferenser per objekt	4–6h
3.9	Drag-and-drop (webb)	Flytta objekt i trädvvy	6–8h
3.10	Ekonomi-flik	Ekonomisk data per objekt	4–6h

Estimat Fas 3: ~60–90 timmar

9. TEKNISKA REKOMMENDATIONER

9.1 Best practices

1. **Behåll mock/live-dualiteten** – Alla nya rutter måste stödja `IS MOCK MODE` med realistisk demodata.
2. **Följ befintlig kodstruktur** – Nya routes följer samma mönster som `orders.ts` :

```

``typescript
// @ts-nocheck (behåll för konsistens med befintliga routes)
import { Router } from 'express';
import { IS MOCK MODE } from '.././proxyHelper';

```

```
const router = Router();
```

```

router.get('/', async (req, res) => {
  if (IS_MOCK_MODE) {
    return res.json({ data: MOCK_OBJECTS, _mock: true });
  }
  // Live-läge: proxy till Plannix
  // ...
});

export { router as objectsRouter };
...

```

1. **Använd `ensure-tables` -mönstret** – Skapa tabeller via SQL `CREATE TABLE IF NOT EXISTS` istället för interaktiv `drizzle-push`.
2. **API-versionering** – Alla nya endpoints under `/api/v1/`.
3. **Svenska felmeddelanden** – Alla användarriktade felmeddelanden på svenska.
4. **OpenAPI-specen** – Uppdatera `lib/api-spec/openapi.yaml` med nya endpoints och kör `pnpm --filter @workspace/api-spec run codegen` för att generera nya React Query-hooks.

9.2 Prestandaoptimering

1. **Rekursiva CTE-queries** – Begränsa djupet (`MAX_DEPTH = 10`) för att undvika oändliga loopar vid cirkulära referenser. Lägg till safeguard:

```

sql
  WITH RECURSIVE object_tree AS (
    SELECT *, 0 AS depth FROM objects WHERE id = $1
    UNION ALL
    SELECT o.*, ot.depth + 1
    FROM objects o JOIN object_tree ot ON o.parent_id = ot.id
    WHERE ot.depth < 10 -- Begränsa djup
  )

```

2. **Paginerig** – Alla list-endpoints stödjer offset-baserad paginerig (LIMIT/OFFSET). Vid >10 000 rader: cursor-baserad paginerig.
3. **Lazy loading av trädnodeer** – Ladda bara första nivån initialt; expandera on-demand.
4. **Index** – Skapa composite indexes för vanliga query-mönster:
 - `(tenant_id, parent_id)` – lista barn
 - `(tenant_id, object_type, status)` – filtrering
 - `(tenant_id, name)` med `pg_trgm` för fuzzy-sökning (valfritt)
5. **Caching** – React Query `staleTime: 30s` (befintligt) räcker. Överväg längre `staleTime` för träddata (60s).
6. **Aggregeringar** – KPI:er beräknas live via COUNT/SUM-queries. Vid prestanda-problem: materialized views eller bakgrundsjobb som uppdaterar sammanfattningstabeller.

9.3 Säkerhetshänsyn

1. **Tenant-isolering** – ALLA queries MÅSTE filtrera på `tenant_id`. Använd en hjälpfunktion:

```

typescript
function withTenantScope(tenantId: string) {

```

```
    return eq(objects.tenantId, tenantId);
  }
```

2. **Rollbaserad åtkomst** – Kundanvändare (`role = 'customer'`) ser ENBART objekt de har åtkomst till via `user_object_access` .

3. **Input-validering** – Använd Zod-scheman (befintligt mönster) för alla request bodies:

```
typescript
const createObjectSchema = z.object({
  name: z.string().min(1).max(500),
  objectNumber: z.string().min(1).max(100),
  objectType: z.enum(['koncern', 'brf', 'fastighet', 'rum', 'karl', 'custom']),
  parentId: z.string().uuid().optional(),
  // ...
});
```

4. **Soft delete** – Ta aldrig bort data permanent; sätt `status = 'archived'` .
5. **Activity log** – Logga alla skrivoperationer i `activity_log` för spårbarhet.
6. **Filuppladdning** – Vinjetbilder och metadata-filer lagras med tenant-scopade paths:
`/uploads/{tenantId}/{objectId}/{filename}`

9.4 Skalbarhet

1. **Horisontell skalning** – Alla nya endpoints är stateless. Socket.IO-rum kan skalas med Redis adapter (befintligt mönster).
2. **Databaspartitionering** – Vid >100 000 objekt per tenant: överväg range partitioning på `tenant_id` .
3. **Sök-prestanda** – Vid >50 000 objekt: överväg PostgreSQL `pg_trgm` -extension eller extern sökmotor (Elasticsearch/Meilisearch).
4. **Bildlagring** – Vinjetbilder och metadata-bilder bör lagras i S3/GCS, inte i databasen. Spara bara URL:en i tabellen.
5. **WebSocket-events** – Utöka befintliga Socket.IO-events för objektändringar:

```
typescript
// Emittera vid objektändring
io.to(`tenant_${tenantId}`).emit('object:updated', { objectId, changes });
io.to(`tenant_${tenantId}`).emit('object:created', { object });
io.to(`tenant_${tenantId}`).emit('object:moved', { objectId, oldParentId, newParentId });
```

10. SAMMANFATTNING

Vad Replit behöver göra (i prioritetsordning):

1. **Skapa databasscheman** – 9 nya tabeller i `lib/db/src/schema/`
2. **Bygga API-endpoints** – ~35 nya endpoints i `artifacts/api-server/src/routes/`
3. **Skapa mock-data** – Realistisk objekthierarki med metadata
4. **Bygga frontend-screens** – ~11 nya screens + ~20 nya komponenter

5. **Uppdatera navigering** – Nya flikar och stack-screens

6. **Uppdatera OpenAPI-spec** – Nya endpoints i `lib/api-spec/openapi.yaml`

7. **Regenerera klienter** – `pnpm --filter @workspace/api-spec run codegen`

Vad som INTE ska ändras:

- **✗** Befintliga 6 DB-tabeller
- **✗** Befintliga mobil-rutter (`/api/v1/mobile/*`)
- **✗** AuthContext / BrandingContext
- **✗** Mock/live-proxy-arkitekturen
- **✗** Socket.IO-setup
- **✗** AsyncStorage-nycklar
- **✗** `react-native-maps` version (1.18.0)

Nyckelprinciper:

- **Additiv utveckling** – Lägg till, ändra inte befintligt
- **Mock-first** – Bygg med mock-data, integrera live senare
- **Tenant-isolering** – Alla queries filterar på `tenant_id`
- **Svenska meddelanden** – Användarriktade strängar på svenska
- **Befintliga mönster** – Följ `@ts-nocheck` , `IS MOCK MODE` , `toV1Path()` , React Query

Dokumentet genererat 2026-06-10 som underlag för Replit AI Agent att implementera utökningarna av Traivo One.